Note matematiche sulle reti neurali

Dal calcolo delle uscite alla discesa del gradiente

Queste note presentano un percorso matematico attraverso il funzionamento e l'allenamento di una rete neurale feed-forward: dalla composizione dei neuroni alla scelta della funzione di errore, dal calcolo delle derivate all'aggiornamento dei parametri. Il riferimento è l'applicazione Dentro la rete: l'obiettivo è comprendere i calcoli e le loro ragioni, entro il caso delle reti qui considerate, senza proporre una trattazione generale dell'apprendimento automatico.

Il percorso principale segue previsione, errore e apprendimento. Le appendici raccolgono approfondimenti sulla temperatura, sul significato informativo dell'entropia incrociata e su alcuni aspetti numerici e implementativi: possono essere consultate nei punti indicati dai rimandi.

Indice

1. Dagli input alla funzione calcolata dalla rete 2
1.1 Una funzione che dipende da parametri 4
1.2 Il calcolo di un neurone 5
1.3 Strati e notazione 5
1.4 Il forward pass 6
2. Il ruolo delle funzioni di attivazione 7
2.1 Perché occorre la non linearità 7
2.2 Sigmoide, tangente iperbolica e ReLU 7
3. Dall'uscita della rete alla funzione di errore 8
3.1 Previsione e decisione 8
3.2 La softmax 9
3.3 La sottrazione del massimo 9
3.4 L'entropia incrociata 9
3.5 Il legame con la massima verosimiglianza 11
4. La discesa del gradiente 11
5. La backpropagation 12
5.1 Il termine δ 12
5.2 Derivata rispetto a un peso e a un bias 13
5.3 Il termine δ nello strato di uscita 13
5.4 Il termine δ in uno strato nascosto 15
5.5 Il procedimento per un singolo esempio 16
6. Come la funzione di errore influisce sul gradiente 16
6.1 Confronto con l'errore quadratico 16
6.2 Un esempio con tre classi 17
7. Dal singolo esempio al dataset 18
7.1 Gradiente medio e campionamento 18
7.2 Epoche e mini-batch 18
8. Dall'allenamento alla generalizzazione 19
8.1 La scala degli input 19
8.2 Perché inizializzare i pesi in modo casuale 19
8.3 Errore di allenamento ed errore su dati nuovi 20
9. Dalle formule a indici alla forma matriciale 20
9.1 Forward 20
9.2 Backward negli strati nascosti 21
9.3 Derivate rispetto ai parametri 21
9.4 Estensione ai mini-batch 22

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Appendice A. Temperatura e distribuzione di Boltzmann 23
Appendice B. Entropia e informazione 24
B.1 Informazione e sorpresa 24
B.2 Entropia di Shannon e divergenza di Kullback–Leibler 24
Appendice C. Il calcolo numerico dell'entropia incrociata 25
Appendice D. Un giro di random reshuffling al primo ordine 26
Appendice E. Corrispondenza con il codice dell'applicazione 27
Riferimenti per gli approfondimenti 27

1. Dagli input alla funzione calcolata dalla rete

Una rete neurale è un modello matematico che serve a produrre previsioni. Impara da esempi già risolti, approssima così la regola che lega gli input al risultato e poi la applica a casi nuovi.

Come esempio prendiamo uno dei problemi del simulatore, la corsa al bus: mancano T minuti alla partenza e ci troviamo a D metri dalla fermata; riusciremo a prendere il bus? Ogni esempio di allenamento è formato da una coppia di input, i valori di T e di D, e dal risultato corrispondente: 1 se il bus è stato preso, 0 se è stato perso. In questo caso la regola è nota (camminando a 80 m/min, il bus si prende quando $D < 80T$) e possiamo quindi confrontarla con quella appresa dalla rete. Nei problemi reali, invece, la regola non è nota o è troppo complicata per essere scritta esplicitamente: è proprio per questo che si ricorre agli esempi. Riconoscere una cifra manoscritta in un'immagine è un problema dello stesso tipo: gli input sono le intensità dei pixel e il risultato è la cifra rappresentata.

Un punto di partenza familiare è l'adattamento di una funzione a dati sperimentali: si sceglie una famiglia di funzioni, per esempio le rette $y = mx + q$, e se ne determinano i parametri, qui m e q, in modo che la funzione approssimi i punti assegnati. Una rete neurale fa la stessa cosa con una famiglia di funzioni molto più ricca, i cui parametri si chiamano pesi e bias. Imparare significa regolare progressivamente questi parametri in base agli esempi di allenamento. Si parla di apprendimento supervisionato perché ogni esempio comprende anche il risultato atteso, che nel seguito chiameremo risultato bersaglio.

Le funzioni della famiglia si costruiscono collegando semplici unità di calcolo, dette neuroni. Il nome è ispirato alla biologia, ma qui indica soltanto un'unità matematica: ogni neurone combina i valori che riceve, secondo i propri pesi e il proprio bias, e produce un nuovo valore (§1.2). I neuroni sono organizzati in strati: il primo riceve gli input, l'ultimo produce i valori di uscita, o output, dai quali si ricava la previsione; gli strati intermedi sono detti nascosti. I valori prodotti da uno strato diventano gli ingressi del successivo, e la funzione calcolata dalla rete è quindi una composizione di funzioni più semplici.

L'allenamento è un ciclo che si ripete molte volte (Figura 1). Si assegnano valori iniziali ai parametri: nell'applicazione i pesi sono scelti a caso e i bias sono posti a zero (§8.2). Per ogni esempio la rete riceve gli input, calcola gli output e ne ricava una previsione, che viene confrontata con il risultato bersaglio mediante una funzione di errore, il cui valore misura quanto la previsione si discosta dal risultato atteso (§3). Le derivate dell'errore rispetto ai pesi e ai bias indicano come modificare ciascun parametro per ridurre l'errore (§4 e §5). I parametri aggiornati diventano i parametri correnti per il passo successivo, e il ciclo viene ripetuto più volte su tutti gli esempi di allenamento (§7). La regola che lega gli input al risultato non viene quindi mai fornita alla rete: viene approssimata attraverso questi aggiornamenti.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

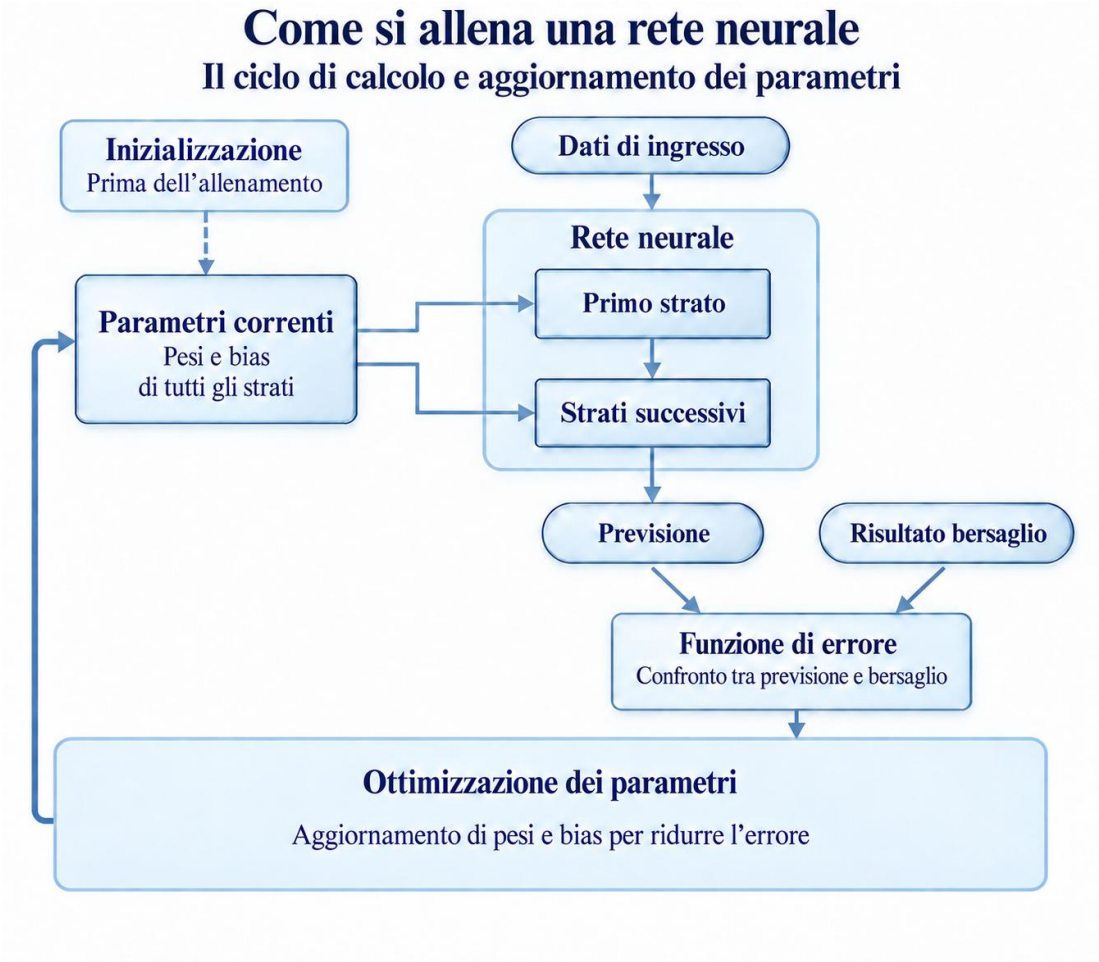


Figura 1. Il ciclo di allenamento. I parametri correnti determinano la funzione calcolata dalla rete; la funzione di errore confronta la previsione con il risultato bersaglio e guida l'aggiornamento dei parametri, che vengono poi usati nel passo successivo.

Terminato l'allenamento, pesi e bias restano fissi e la rete si usa come una qualunque funzione: riceve nuovi input e calcola gli output con i parametri appresi (Figura 2). Nel problema del bus, per esempio, la rete allenata fornisce una risposta anche per coppie di valori di T e D che non comparivano fra gli esempi. Un buon accordo con gli esempi di allenamento non garantisce però, da solo, previsioni corrette su dati nuovi: questa capacità, detta generalizzazione, va verificata separatamente, su esempi non usati durante l'allenamento (§8.3).

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

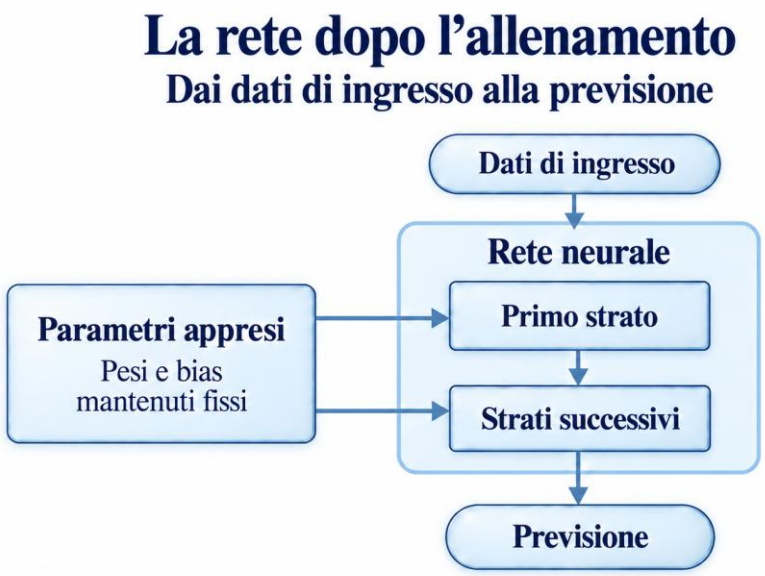


Figura 2. La rete dopo l'allenamento. Con i parametri appresi mantenuti fissi, la rete trasforma i dati di ingresso in una previsione: non ci sono più né il confronto con un risultato bersaglio né l'aggiornamento dei parametri.

Nel simulatore la rete è volutamente piccola, in modo che si possano osservare in dettaglio pesi, bias, valori calcolati dai neuroni ed errore. Le reti usate nelle applicazioni reali seguono gli stessi principi, con un numero di parametri enormemente maggiore.

1.1 Una funzione che dipende da parametri

Fissata la struttura della rete (quanti strati, quanti neuroni in ciascuno strato e quali funzioni di attivazione: §1.2 e §2), la funzione calcolata dalla rete dipende soltanto dai valori dei parametri. La indichiamo con $F_\theta(x)$: x è il vettore degli input, θ il vettore che raccoglie tutti i pesi e i bias. Se l'uscita è un solo numero, indichiamo la previsione con $\hat{y} = F_\theta(x)$; nei problemi con più classi l'uscita è invece un vettore, con una componente per ciascuna classe (§3). Al variare di θ si ottiene una famiglia di funzioni, così come al variare di m e q si ottiene la famiglia delle rette $y = mx + q$.

La distinzione fra x e θ permette di descrivere con precisione le due fasi appena introdotte, nelle quali le due variabili si scambiano i ruoli. Nell'uso della rete allenata θ è fisso e varia x: F_θ è una funzione degli input (Figura 2). Durante l'allenamento sono invece fissati gli esempi e varia θ: l'errore commesso sugli esempi di allenamento diventa una funzione dei soli parametri. Se gli esempi sono N e $E_r(\theta)$ è l'errore sull'esempio r, si considera l'errore medio

$$\bar{E}(\theta) = \frac{1}{N}\sum_{r=1}^{N} E_r\,(\theta).$$

Allenare la rete significa cercare valori di θ per cui $\bar{E}(\theta)$ sia piccolo, cioè scegliere all'interno della famiglia una funzione che si accordi bene con gli esempi. Gli stessi pesi e bias valgono per tutti gli esempi: si cerca una sola funzione, non una funzione diversa per ciascun esempio.

Per la retta che approssima un insieme di punti nel senso dei minimi quadrati, i coefficienti migliori si ricavano con una formula esplicita. Per una rete neurale questo in generale non è possibile: si parte dai valori iniziali dei parametri e li si migliora per passi successivi, usando le derivate di $\bar{E}$ rispetto ai parametri. È il metodo della discesa del gradiente (§4); la backpropagation (§5) è il procedimento che permette di calcolare queste derivate in modo efficiente.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Nelle reti considerate in questa dispensa, dette feed-forward, le connessioni vanno sempre da uno strato al successivo, dagli input verso l'uscita: F_θ è quindi la composizione delle trasformazioni calcolate dagli strati successivi, come mostrato nella Figura 3 del §1.4.

1.2 Il calcolo di un neurone

Un neurone combina i valori ricevuti con una somma pesata, aggiunge un termine costante detto bias e applica una funzione di attivazione. Nel caso di due input:

$$z = w_1 x_1 + w_2 x_2 + b, \qquad a = f(z).$$

Il valore z è detto pre-attivazione; a è l'attivazione trasmessa ai neuroni successivi. I pesi stabiliscono come ciascun ingresso contribuisce a z; il bias ne modifica il valore anche a input fissati. Il nome "neurone" designa qui questa semplice unità di calcolo, senza pretendere alcun significato biologico.

La prima operazione è affine: è una trasformazione lineare più un termine costante. Con due input, se almeno un peso è diverso da zero, l'equazione $w_1 x_1 + w_2 x_2 + b = 0$ descrive una retta. Se l'attivazione è la sigmoide (§2.2) e la soglia di classificazione è $1/2$, questa retta separa le due regioni di decisione, perché $\sigma(0) = \frac{1}{2}$ e la sigmoide è crescente, per cui $z < 0$ implica $\sigma(z) < \frac{1}{2}$ e $z > 0$ implica $\sigma(z) > \frac{1}{2}$.

1.3 Strati e notazione

Numeriamo gli strati da 0 a L. Lo strato 0 contiene gli input; lo strato L è lo strato di uscita. Indichiamo con n_ℓ il numero di nodi dello strato ℓ; i nodi dello strato ℓ sono numerati da 1 a n_ℓ.

Indichiamo con $x = (x_1, \dots, x_{n_0})^T$ il vettore colonna che raccoglie gli input del singolo esempio e poniamo $a_i^{(0)} = x_i$. Useremo inoltre la seguente convenzione:

Simbolo	Significato
$a_i^{(\ell)}$	valore del nodo i dello strato ℓ; per $\ell = 0$, $a_i^{(0)} = x_i$
$w_{ij}^{(\ell)}$	peso della connessione dal nodo j dello strato $\ell - 1$ al neurone i dello strato ℓ
$b_i^{(\ell)}$	bias del neurone i dello strato ℓ
$z_i^{(\ell)}$	valore pre-attivazione del neurone i dello strato ℓ
f_ℓ	funzione di attivazione usata nello strato ℓ

Per $a_i^{(\ell)}$ è ammesso anche $\ell = 0$; per pesi, bias, pre-attivazioni e funzioni di attivazione si intende $\ell = 1, \dots, L$. I passaggi tra strati vanno quindi da $\ell - 1$ a ℓ, con $\ell = 1, \dots, L$.

Nei problemi con una sola uscita, y indica il valore bersaglio; nel caso multiclasse c indica l'indice della classe corretta e t_i le componenti del bersaglio *one-hot* (1 per la classe corretta, 0 per tutte le altre).

Gli strati tra l'ingresso e l'uscita sono detti nascosti. Per le loro attivazioni non è assegnato un valore bersaglio nei dati: i parametri vengono modificati in base all'errore finale della rete. Nelle reti considerate ogni neurone riceve tutte le attivazioni dello strato precedente; si parla di strati completamente connessi.

Nella notazione $w_{ij}^{(\ell)}$, l'indice i identifica il neurone di arrivo, j quello di partenza e ℓ lo strato di arrivo. Per esempio, $w_{21}^{(2)}$ collega il primo neurone del primo strato nascosto al secondo del successivo. L'apice tra parentesi indica lo strato, non una potenza. La corrispondenza con gli array del codice è riportata nel §E.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Nel pannello “Per cominciare” dell’applicazione, per descrivere un singolo neurone si sottintendono gli indici di strato e di neurone d’arrivo, mentre le connessioni in ingresso sono numerate con l’indice i: il calcolo si scrive quindi $z = \sum_i w_i\, a_i + b$. L’indice i del pannello corrisponde quindi all’indice j usato in questa dispensa.

1.4 Il forward pass

Per $\ell = 1, \dots, L$, il neurone i dello strato ℓ riceve i valori $a_j^{(\ell-1)}$ da tutti i nodi dello strato precedente, ne calcola prima la somma pesata

$$z_i^{(\ell)} = \sum_{j=1}^{n_{\ell-1}} w_{ij}^{(\ell)}\, a_j^{(\ell-1)} + b_i^{(\ell)} \tag{1}$$

e, negli strati in cui l’attivazione è applicata componente per componente, applica poi la funzione di attivazione:

$$a_i^{(\ell)} = f_\ell\big(z_i^{(\ell)}\big). \tag{2}$$

Nel caso multiclasse dell’applicazione, nello strato di uscita viene invece applicata la softmax all’intero vettore dei valori pre-attivazione. Questa funzione sarà presentata nel §3.2. Il forward pass, cioè il calcolo dell’uscita della rete a partire dagli input, ripete queste due operazioni strato dopo strato fino all’uscita. Per un problema con una sola uscita, quest’ultima è

$$\hat{y} = a_1^{(L)}.$$

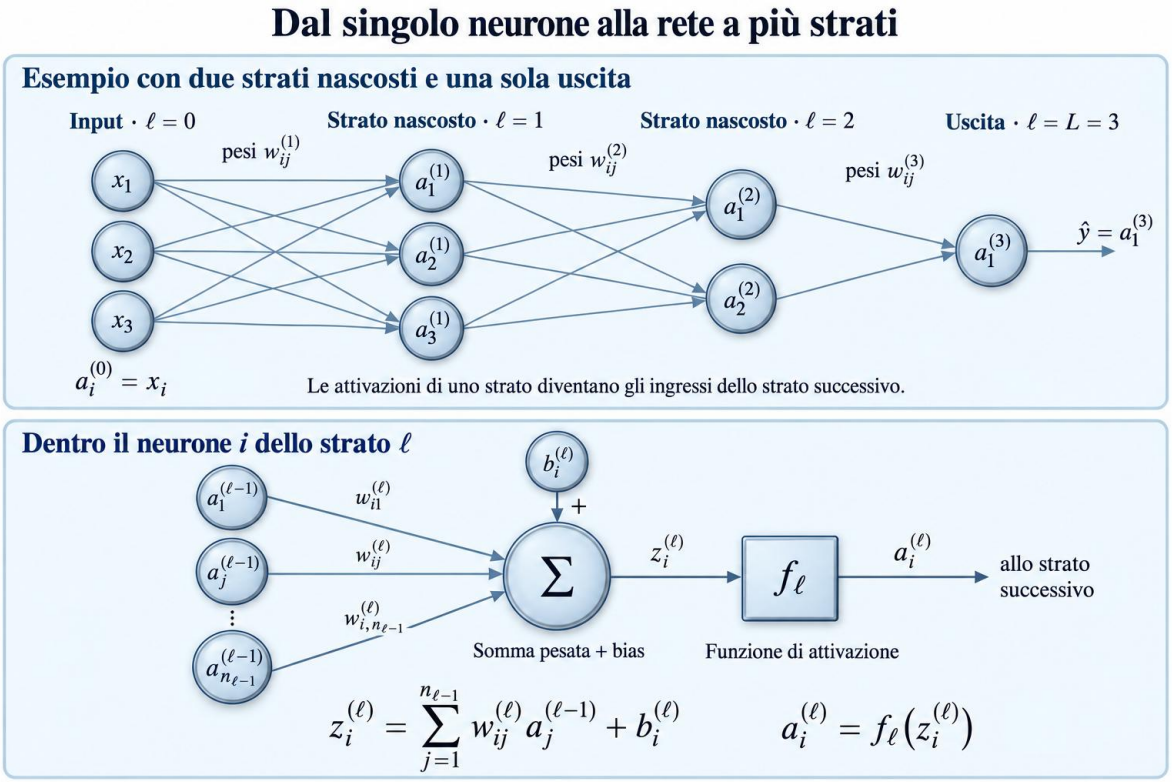


Figura 3. Rete con tre input, due strati nascosti e una sola uscita. In basso è esplicitato il calcolo del generico neurone. I bias, omessi nello schema superiore per leggibilità, sono presenti in ogni neurone nascosto e di uscita. La trasformazione softmax del §3.2 riguarda invece l’intero vettore di uscita.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

2. Il ruolo delle funzioni di attivazione

2.1 Perché occorre la non linearità

Aumentare il numero di strati, da solo, non basta a ottenere funzioni più generali. Se tutte le attivazioni degli strati nascosti fossero l'identità $f(z) = z$, una composizione di trasformazioni affini resterebbe affine. Lo si vede già con due strati, scrivendo le somme pesate in forma matriciale:

$$v = Ax + b, \qquad u = Cv + d = CAx + (Cb + d).$$

Qui A e C sono matrici di pesi, b e d vettori di bias; x, v e u sono vettori di valori. Il risultato ha ancora la forma "matrice per input più vettore costante". Ripetendo il ragionamento, tutti gli strati affini si possono ricondurre a una sola trasformazione affine.

Un'eventuale sigmoide o softmax finale non elimina questa limitazione degli strati nascosti: nella classificazione binaria la frontiera resta un iperpiano (una retta nel caso di due input); nel caso multiclasse le uguaglianze fra coppie di pre-attivazioni di uscita sono ancora equazioni affini. Le attivazioni non lineari negli strati nascosti consentono invece di costruire trasformazioni per le quali questa riduzione non è possibile.

2.2 Sigmoide, tangente iperbolica e ReLU

L'applicazione permette di confrontare tre attivazioni negli strati nascosti, i cui grafici sono riportati nella Figura 4. Le loro derivate entreranno direttamente nelle formule della backpropagation (§5).

Funzione	Definizione	Derivata
Sigmoide	$\sigma(z) = \dfrac{1}{1+e^{-z}}$	$\sigma'(z) = \sigma(z)(1-\sigma(z))$
Tangente iperbolica	$\tanh(z) = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$\tanh'(z) = 1-\tanh^{2}(z)$
ReLU	$f(z) = \max(0, z)$	0 per $z < 0$; 1 per $z > 0$

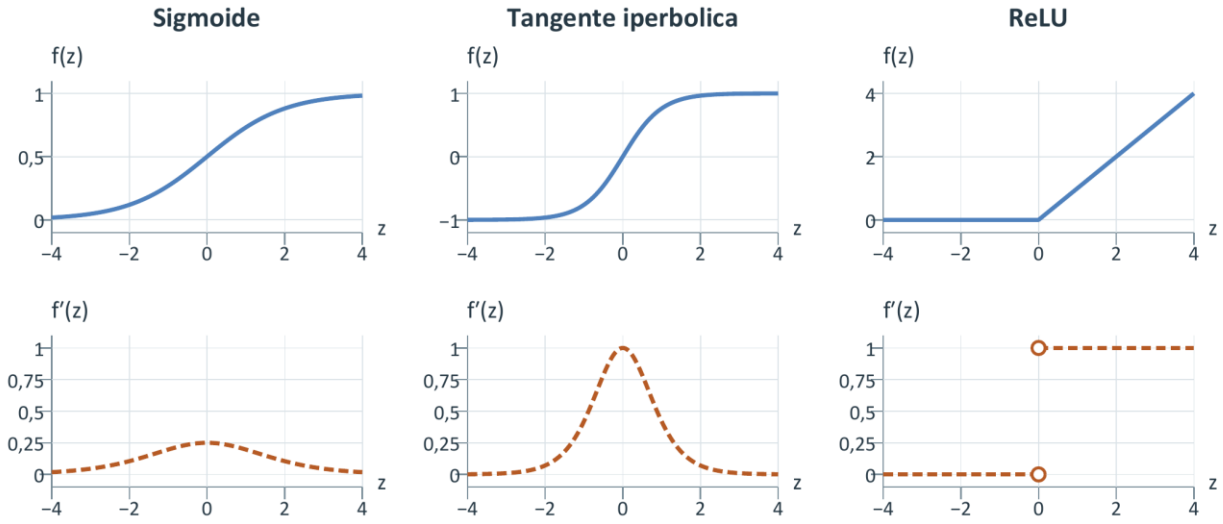


Figura 4. Funzioni di attivazione (in alto) e rispettive derivate (in basso). L'intervallo di z è comune a tutti i grafici; anche la scala verticale è comune alle tre derivate. Per la ReLU, i cerchi vuoti indicano che la derivata non è definita in $z = 0$.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

La sigmoide assume valori in $(0,1)$, la tangente iperbolica in $(-1,1)$. Le due funzioni sono legate dall'identità $\tanh(z) = 2\sigma(2z) - 1$: il grafico della tangente iperbolica si ottiene da quello della sigmoide mediante un riscalamento degli assi e una traslazione verticale. Entrambe si appiattiscono quando $|z|$ è grande: le derivate tendono a zero e si parla di saturazione. La tangente iperbolica, a differenza della sigmoide, permette anche attivazioni negative.

La ReLU (rectified linear unit) restituisce zero per gli argomenti negativi e lascia invariati quelli positivi. È affine a tratti, ma non è una funzione affine sull'intero dominio: proprio il cambio di espressione consente di costruire trasformazioni più articolate. In $z = 0$ non è derivabile; nel codice dell'applicazione si usa convenzionalmente il valore zero al posto della derivata. Le formule di derivazione vanno quindi intese dove le derivate esistono, con questa convenzione nei punti di non derivabilità.

La scelta dell'attivazione influisce sia sulle funzioni rappresentabili sia sul calcolo dei gradienti. La saturazione può attenuare le derivate propagate attraverso più strati; una ReLU, invece, ha derivata unitaria per $z > 0$, ma non trasmette il gradiente attraverso il ramo $z < 0$. Se la pre-attivazione è negativa per tutti gli esempi di allenamento, i gradienti rispetto ai pesi entranti e al bias sono nulli: finché questa condizione persiste, la discesa del gradiente non modifica questi parametri. Negli strati successivi al primo, tuttavia, gli aggiornamenti degli strati precedenti possono cambiare gli ingressi del neurone e riattivarlo. Non esiste dunque una scelta che renda automaticamente semplice l'allenamento in ogni situazione.

3. Dall'uscita della rete alla funzione di errore

3.1 Previsione e decisione

La trasformazione dello strato di uscita dipende dal tipo di problema. La tabella riassume le scelte dell'applicazione; la trasformazione di uscita può essere diversa da quella degli strati nascosti.

Problema	Uscita della rete	Interpretazione e decisione
Regressione	Un valore $\hat{y} = z_1^{(L)}$	Previsione numerica, senza soglia di classificazione
Classificazione binaria	Un valore $a = \sigma(z_1^{(L)})$	Stima della probabilità della classe 1; classe 1 se $a \geq 1/2$, classe 0 altrimenti
Classificazione multiclasse	Un vettore p prodotto dalla softmax	Stime di probabilità per le classi; scelta di una componente massima

Nei problemi con una sola uscita useremo per brevità anche $a = \hat{y} = a_1^{(L)}$ e $z = z_1^{(L)}$: a e $\hat{y}$ indicano quindi lo stesso valore, mentre y resta il bersaglio.

La soglia e la scelta del massimo traducono le uscite in una decisione. La funzione di errore, o loss function, confronta invece l'uscita numerica con il bersaglio e guida l'allenamento. Per regressione e classificazione binaria, l'applicazione usa l'errore quadratico del singolo esempio:

$$E = \frac{1}{2}(\hat{y} - y)^2.$$

Il fattore $1/2$ semplifica la derivata, senza cambiare i punti di minimo. Nella regressione l'errore non ha un limite superiore, mentre nel caso binario, con $\hat{y} \in [0,1]$ e $y \in \{0,1\}$, è al massimo $1/2$. Per la classificazione multiclasse sarà usata invece l'entropia incrociata; il confronto fra le due scelte, compresa la diversa convenzione per l'errore quadratico vettoriale, sarà ripreso nel §6.1.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

3.2 La softmax

Nel caso multiclasse, lo strato di uscita contiene $K = n_L$ neuroni, uno per classe. Indichiamo per brevità con $z_i = z_i^{(L)}$ le pre-attivazioni di uscita, spesso chiamate logits. La softmax li trasforma nelle componenti

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \qquad i = 1, \dots, K.$$

Ogni esponenziale è positivo; dividendo ciascuno di essi per la loro somma si ottiene $p_i > 0$ e $\sum_{i=1}^{K} p_i = 1$. Ogni uscita dipende da tutte le pre-attivazioni, perché il denominatore è comune. La softmax è quindi una trasformazione dell'intero vettore, come anticipato nel §1.4.

L'ordine dei valori si conserva: se $z_i > z_j$, allora $p_i > p_j$. Più precisamente,

$$\frac{p_i}{p_j} = e^{z_i - z_j}.$$

Le probabilità relative dipendono dunque dalle differenze fra le pre-attivazioni: aggiungere la stessa costante a tutte le componenti non cambia la distribuzione. La softmax restituisce tutte le probabilità, e l'applicazione individua poi la classe prevista scegliendo quella con probabilità massima.

Per esempio, con $z = (2; 0; -1)$ si ottiene $p \approx (0{,}843795; 0{,}114195; 0{,}042010)$. Il §6.2 riprende questi valori per calcolare l'errore e il gradiente.

Le componenti della softmax hanno quindi le proprietà matematiche di una distribuzione di probabilità, ma non riflettono necessariamente la frequenza reale di previsioni corrette: tra tutte le previsioni fatte con confidenza 0,9, ad esempio, la quota di corrette può essere ben diversa dal 90%.

3.3 La sottrazione del massimo

La formula precedente può essere problematica al calcolatore: per valori positivi grandi, l'esponenziale può superare il massimo numero rappresentabile (overflow). Per evitarlo si pone $z_{\max} = \max(z_1, \dots, z_K)$ e si usa la forma equivalente

$$p_i = \frac{e^{z_i - z_{\max}}}{\sum_{j=1}^{K} e^{z_j - z_{\max}}} = \frac{e^{-z_{\max}} e^{z_i}}{e^{-z_{\max}} \sum_{j=1}^{K} e^{z_j}} = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}},$$

usata anche nell'applicazione. In questo modo tutti gli esponenti risultano non positivi e almeno uno è zero: gli esponenziali sono al massimo 1 e la loro somma è compresa tra 1 e K. Per esempio, da $z = (1000; 998; 997)$ si passa a $(0; -2; -3)$, ottenendo le stesse probabilità dell'esempio precedente senza calcolare e^{1000}.

Questa forma non impedisce però che probabilità p_i molto piccole siano arrotondate a zero (underflow). Il §C spiega come ciò influisce sul calcolo dell'entropia incrociata. Per l'analisi numerica si veda [1].

Una variante della softmax, non utilizzata nell'applicazione, introduce anche una temperatura che modifica la concentrazione delle probabilità. Il §A ne descrive il significato e il legame formale con la distribuzione di Boltzmann.

3.4 L'entropia incrociata

Consideriamo ora un problema in cui ogni esempio appartiene a una sola tra K classi. Nello strato di uscita la rete applica la softmax ai valori pre-attivazione e produce un vettore $p = (p_1, \dots, p_K)$: ogni componente è positiva e la loro somma vale 1. Come scritto in precedenza, questi valori esprimono le probabilità stimate dal modello per le diverse classi e nel loro insieme costituiscono una *distribuzione di probabilità* sulle classi.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Indichiamo con c la classe corretta dell'esempio. La rappresentiamo con un vettore bersaglio $t = (t_1, \dots, t_K)$, detto *one-hot*: $t_c = 1$ e tutte le altre componenti sono nulle. Anche t è quindi una distribuzione di probabilità, concentrata sulla classe osservata.

Per allenare la rete con la discesa del gradiente, non basta contare le classificazioni sbagliate: se, per esempio, una piccola variazione dei parametri aumentasse la probabilità assegnata alla classe corretta senza renderla ancora la più probabile, il conteggio delle attribuzioni errate non rileverebbe questo miglioramento. Al variare dei parametri, infatti, il conteggio delle classificazioni errate rimane costante a tratti e cambia per salti: il suo gradiente è quindi nullo nei tratti costanti e non definito nei punti di salto. Serve quindi una funzione di errore che tenga conto delle probabilità assegnate e fornisca, attraverso il gradiente, un'indicazione su come aggiornare i parametri.

Per misurare l'errore possiamo associare alla classe osservata la penalità $-\ln p_c$: è piccola quando p_c è vicino a 1 e cresce quando p_c diminuisce. Le ragioni per scegliere proprio il logaritmo saranno sviluppate nel §3.5.

Questa funzione è un caso particolare dell'*entropia incrociata* (*cross-entropy*). La definizione generale si applica a una qualunque distribuzione bersaglio t, con componenti non negative e somma 1. Si associa a ciascuna classe q la penalità $-\ln p_q$ e se ne calcola la media ponderata, usando come pesi le probabilità t_q della distribuzione bersaglio:

$$E = -\sum_{q=1}^{K} t_q \ln p_q. \tag{3}$$

Quindi p_q determina la penalità associata alla classe q, mentre t_q ne stabilisce il peso nella media.

Nel nostro problema, il bersaglio one-hot dà peso 1 alla classe osservata e peso 0 a tutte le altre. La media ponderata si riduce così a un solo termine:

$$E = -\ln p_c. \tag{4}$$

La forma generale (3) è anche il punto di partenza della derivazione del corrispondente segnale d'errore $\delta = p - t$ nel §5.3.3.

L'errore del singolo esempio dipende dunque soltanto da p_c. Tuttavia, le probabilità prodotte dalla softmax sono legate dal vincolo $\sum_i p_i = 1$: una variazione della probabilità complessivamente assegnata alle altre classi comporta quindi anche una variazione opposta di p_c. Una redistribuzione tra le altre classi che ne mantenga invariata la somma lascia invece invariati p_c ed E.

La funzione di errore $-\ln p_c$ è decrescente. Con almeno due classi e valori pre-attivazione finiti, la softmax produce $0 < p_c < 1$, per cui l'errore è positivo, tende a $+\infty$ quando p_c tende a 0^+ e tende a 0 quando p_c tende a 1. L'errore nullo è quindi un valore limite.

p_c	$E = -\ln p_c$
0,99	0,010
0,9	0,105
0,5	0,693
0,1	2,303
0,01	4,605

Assegnare una probabilità molto bassa alla classe osservata comporta quindi una penalità elevata: è ciò che accade, in particolare, quando il modello attribuisce una probabilità vicina a 1 a una classe sbagliata. In generale, ogni volta che p_c si riduce di un fattore 10, l'errore aumenta di $\ln 10 \approx 2{,}303$.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Va notato che l'entropia incrociata e la percentuale di classificazioni corrette misurano aspetti diversi: ad esempio, se p_c passa da $0{,}6$ a $0{,}9$, la classe prevista resta corretta, ma l'errore scende da circa $0{,}511$ a $0{,}105$. Durante l'allenamento, un aumento delle probabilità assegnate alle classi corrette può quindi ridurre l'errore anche se il numero di classificazioni corrette rimane invariato.

3.5 Il legame con la massima verosimiglianza

Consideriamo N esempi di allenamento, con input x_r e classe osservata c_r. Indichiamo con θ l'insieme dei pesi e dei bias e con $p_{c_r}(x_r;\theta)$ la probabilità che la rete assegna all'etichetta osservata dell'esempio r.

Fissati gli input e i parametri del modello, assumiamo che le etichette dei diversi esempi siano indipendenti. La probabilità che il modello assegna all'intera sequenza di etichette osservate si ottiene allora moltiplicando le probabilità dei singoli esempi:

$$\mathcal{L}(\theta) = \prod_{r=1}^{N} p_{c_r}(x_r;\theta). \tag{5}$$

Gli esempi e le loro etichette restano fissi; variando i parametri θ, cambiano le probabilità assegnate dalla rete e quindi il prodotto $\mathcal{L}(\theta)$. Considerata come funzione dei parametri, questa quantità è detta *verosimiglianza*: il criterio di massima verosimiglianza consiste nel cercare i parametri che la massimizzano.

Poiché il logaritmo è crescente, massimizzare $\mathcal{L}$ equivale a massimizzare $\ln\mathcal{L}$. Cambiando segno e dividendo per N, otteniamo l'obiettivo da minimizzare:

$$\overline{E}(\theta) = -\frac{1}{N}\ln\mathcal{L}(\theta) = -\frac{1}{N}\sum_{r=1}^{N}\ln\, p_{c_r}(x_r;\theta) = \frac{1}{N}\sum_{r=1}^{N} E_r(\theta). \tag{6}$$

Minimizzare l'entropia incrociata media equivale quindi a massimizzare la verosimiglianza delle etichette nel modello. L'applicazione cerca di ridurre questo errore medio (§7) aggiornando i parametri dopo ogni esempio.

Oltre a questo, il logaritmo ha anche un'interpretazione in termini di informazione associata a un evento: il §B sviluppa questa lettura, introduce l'entropia di Shannon e chiarisce il nome "entropia incrociata". Questi concetti completano l'interpretazione della funzione loss, ma non sono necessari per ricavarne il gradiente.

4. La discesa del gradiente

Scelta la funzione di errore, resta da capire come modificare i parametri. Per un dataset di N esempi, indichiamo con $E_r(\theta)$ l'errore dell'esempio r e con $\overline{E}(\theta)$ la media:

$$\bar{E}(\theta) = \frac{1}{N}\sum_{r=1}^{N} E_r(\theta),$$

dove raccogliamo nuovamente i parametri in un vettore $\theta = (\theta_1, \dots, \theta_P)$. Il gradiente $g = \nabla\overline{E}(\theta)$ è il vettore delle derivate parziali: la sua componente j indica la variazione locale dell'errore medio rispetto al parametro θ_j, mantenendo fissi gli altri. Se la funzione è differenziabile nel punto considerato, per uno spostamento piccolo h vale l'approssimazione del primo ordine

$$\overline{E}(\theta + h) = \overline{E}(\theta) + g^T h + o(\|h\|).$$

Qui $g^T h = \sum_{j=1}^{P} g_j\, h_j$ è il prodotto scalare, $\|h\|$ è la lunghezza del vettore spostamento e il simbolo $o(\|h\|)$ indica un resto che, diviso per $\|h\|$, tende a zero quando h tende al vettore nullo.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Ponendo $h = -\eta g$, a punto θ fissato e per $\eta \to 0^+$, si ottiene

$$\overline{E}(\theta - \eta g) = \overline{E}(\theta) - \eta \|g\|^2 + o(\eta).$$

Per $g \neq 0$ e η sufficientemente piccolo, il termine negativo prevale sul resto e l'errore diminuisce. Il numero positivo η è il tasso di apprendimento: controlla l'ampiezza del passo. Un passo troppo grande può aumentare l'errore, perché l'approssimazione locale perde validità; uno molto piccolo può rendere lento l'allenamento. Nei punti in cui il gradiente è nullo questo argomento non fornisce una direzione di diminuzione e non garantisce che si sia raggiunto un minimo.

Per un singolo esempio si applica la stessa regola alla sua loss E. Il generico parametro θ_j, che può essere un peso o un bias, viene aggiornato secondo

$$\theta_{j,\text{nuovo}} = \theta_j - \eta \frac{\partial E}{\partial \theta_j}, \tag{7}$$

con tutte le derivate calcolate ai valori correnti dei parametri. Ridurre l'errore di un esempio non implica ridurre a ogni passo l'errore medio del dataset: questa distinzione sarà esaminata nel §7.

La discesa del gradiente stabilisce come usare le derivate per aggiornare i parametri. La backpropagation stabilisce invece come calcolarle in modo efficiente, applicando la regola della catena dall'uscita verso gli strati precedenti e riutilizzando derivate già ottenute. Sono due operazioni distinte dello stesso processo di allenamento.

5. La backpropagation

5.1 Il termine δ

Come abbiamo visto nell'equazione (7), per aggiornare pesi e bias mediante la discesa del gradiente, servono le derivate parziali

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} \quad \text{e} \quad \frac{\partial E}{\partial b_i^{(\ell)}}.$$

Per organizzare questo calcolo, associamo a ogni neurone una quantità che verrà riutilizzata nel calcolo a ritroso delle derivate (*backward pass*):

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial z_i^{(\ell)}}.$$

Il simbolo $\delta_i^{(\ell)}$, spesso chiamato informalmente **segnale d'errore**, rappresenta quindi la derivata dell'errore E rispetto al valore pre-attivazione $z_i^{(\ell)}$ del neurone corrispondente; esprime quindi la sensibilità locale, con segno, dell'errore rispetto a variazioni della pre-attivazione.

Quando l'attivazione è applicata componente per componente, poiché

$$a_i^{(\ell)} = f_\ell\big(z_i^{(\ell)}\big),$$

la regola della catena dà anche

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial a_i^{(\ell)}} \, f_\ell'\big(z_i^{(\ell)}\big).$$

Nel paragrafo seguente useremo $\delta_i^{(\ell)}$ per ricavare le derivate rispetto ai pesi e al bias che entrano nel neurone.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

5.2 Derivata rispetto a un peso e a un bias

Consideriamo un peso preciso, $w_{ij}^{(\ell)}$. Nella somma

$$z_i^{(\ell)} = \sum_{q=1}^{n_{\ell-1}} w_{iq}^{(\ell)}\, a_q^{(\ell-1)} + b_i^{(\ell)},$$

quel peso compare soltanto nel termine $w_{ij}^{(\ell)} a_j^{(\ell-1)}$. Quindi

$$\frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}} = a_j^{(\ell-1)}.$$

Applicando la regola della catena:

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \frac{\partial E}{\partial z_i^{(\ell)}} \frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}. \tag{8}$$

Per il bias,

$$\frac{\partial z_i^{(\ell)}}{\partial b_i^{(\ell)}} = 1,$$

e dunque

$$\frac{\partial E}{\partial b_i^{(\ell)}} = \delta_i^{(\ell)}. \tag{9}$$

Le due relazioni si possono quindi riassumere nella forma:

$$\frac{\partial E}{\partial w} = \delta_{\text{arrivo}}\, a_{\text{partenza}}, \qquad \frac{\partial E}{\partial b} = \delta_{\text{neurone}}.$$

5.3 Il termine δ nello strato di uscita

I valori di $\delta_i^{(L)}$ per i neuroni in uscita si ricavano analiticamente dalla funzione di errore e dalla trasformazione di uscita: in questo paragrafo vedremo in dettaglio i casi implementati nell'applicazione. Negli strati nascosti, invece, i valori di $\delta_i^{(\ell)}$ vengono calcolati ricorsivamente a partire da quelli dello strato successivo, come verrà descritto nel §5.4.

5.3.1 Sigmoide ed errore quadratico

Con una sola uscita, utilizzando sigmoide ed errore quadratico come nell'applicazione, si ha

$$a = \hat{y} = \sigma(z), \qquad E = \frac{1}{2}(a - y)^2.$$

Poiché

$$\frac{\partial E}{\partial a} = a - y \qquad \text{e} \qquad \sigma'(z) = a(1 - a),$$

si ottiene

$$\delta^{(L)} = (a - y)\, a(1 - a). \tag{10a}$$

Nella pratica, tuttavia, per la classificazione binaria è comune associare la sigmoide all'entropia incrociata binaria; in quel caso il termine di uscita si semplifica in $\delta^{(L)} = a - y$. Il confronto fra le due funzioni di errore è sviluppato nel §6.1.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

5.3.2 Uscita lineare ed errore quadratico

Se l'ultimo strato usa $a = z$, allora $f'(z) = 1$ e quindi

$$\delta^{(L)} = a - y\,. \tag{10b}$$

5.3.3 Softmax ed entropia incrociata

Usiamo ora la softmax del §3.2 e la funzione di errore del §3.4 per ricavare le derivate rispetto alle pre-attivazioni di uscita.

Se $z_i^{(L)}$ è il valore pre-attivazione del neurone di uscita i, la softmax produce

$$p_i = \frac{e^{z_i^{(L)}}}{\sum_{k=1}^{n_L} e^{z_k^{(L)}}}.$$

Utilizziamo la codifica one-hot, in cui indichiamo con t_i il valore bersaglio: $t_i = 1$ per la classe corretta e $t_i = 0$ per tutte le altre. L'entropia incrociata (*cross-entropy*) del singolo esempio è

$$E = -\sum_{q=1}^{n_L} t_q \ln p_q = -\ln p_c,$$

dove c è la classe corretta. Nella softmax ogni probabilità dipende da tutte le pre-attivazioni di uscita. Applicando la regola della catena attraverso le probabilità, occorrerebbe tenere conto di questa dipendenza. Il calcolo si semplifica esprimendo prima l'errore direttamente in funzione delle pre-attivazioni. Poiché

$$\ln p_q = z_q^{(L)} - \ln\left(\sum_{k=1}^{n_L} e^{z_k^{(L)}}\right)$$

e $\sum_i t_i = 1$, possiamo riscrivere

$$E = -\sum_{q=1}^{n_L} t_q z_q^{(L)} + \ln\left(\sum_{k=1}^{n_L} e^{z_k^{(L)}}\right).$$

Derivando rispetto a $z_i^{(L)}$ si ottiene quindi

$$\delta_i^{(L)} = \frac{\partial E}{\partial z_i^{(L)}} = -t_i + \frac{e^{z_i^{(L)}}}{\sum_{k=1}^{n_L} e^{z_k^{(L)}}} = p_i - t_i. \tag{10c}$$

Per la classe corretta si ha quindi $\delta_c^{(L)} = p_c - 1 < 0$; per ogni altra classe si ha $\delta_i^{(L)} = p_i > 0$. Considerando i valori $z_i^{(L)}$ come variabili, la direzione di discesa aumenta quindi quello della classe corretta e diminuisce quelli delle altre classi.

Per i parametri dello strato di uscita valgono le relazioni

$$\frac{\partial E}{\partial w_{ij}^{(L)}} = \delta_i^{(L)} a_j^{(L-1)}, \qquad \frac{\partial E}{\partial b_i^{(L)}} = \delta_i^{(L)}.$$

Gli aggiornamenti si ottengono moltiplicando queste derivate per $-\eta$. Anche se ogni componente di $\delta^{(L)}$ appartiene a $(-1,1)$, l'ampiezza dell'aggiornamento dipende quindi dal tasso di apprendimento e, per i pesi, dalle attivazioni dello strato precedente.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

5.4 Il termine δ in uno strato nascosto

Consideriamo il neurone i di uno strato nascosto ℓ. La sua attivazione $a_i^{(\ell)}$ raggiunge tutti i neuroni $k = 1, \dots, n_{\ell+1}$ dello strato successivo. Per ciascuno di essi,

$$z_k^{(\ell+1)} = \sum_{q=1}^{n_\ell} w_{kq}^{(\ell+1)} a_q^{(\ell)} + b_k^{(\ell+1)}.$$

Vogliamo calcolare

$$\delta_i^{(\ell)} = \frac{\partial E}{\partial z_i^{(\ell)}}.$$

L'errore dipende da $z_i^{(\ell)}$ attraverso l'attivazione $a_i^{(\ell)}$ e poi attraverso tutti i neuroni successivi. La regola della catena deve quindi sommare tutti questi percorsi:

$$\frac{\partial E}{\partial a_i^{(\ell)}} = \sum_{k=1}^{n_{\ell+1}} \frac{\partial E}{\partial z_k^{(\ell+1)}} \frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}}.$$

Ma

$$\frac{\partial E}{\partial z_k^{(\ell+1)}} = \delta_k^{(\ell+1)} \qquad \text{e} \qquad \frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}} = w_{ki}^{(\ell+1)},$$

perciò

$$\frac{\partial E}{\partial a_i^{(\ell)}} = \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \, \delta_k^{(\ell+1)}.$$

Infine, $a_i^{(\ell)} = f_\ell\big(z_i^{(\ell)}\big)$, quindi

$$\delta_i^{(\ell)} = f_\ell{}'\big(z_i^{(\ell)}\big) \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)} \delta_k^{(\ell+1)}. \tag{11}$$

Questa formula ricorsiva permette quindi di calcolare il termine δ di uno strato nascosto a partire dai termini δ dello strato successivo (Figura 5).

In forma mnemonica, la struttura della ricorrenza è quindi:

$$\delta_{\text{neurone}} = f'(z_{\text{neurone}}) \cdot \sum_{\text{successivi}} (w \cdot \delta_{\text{successivo}}).$$

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

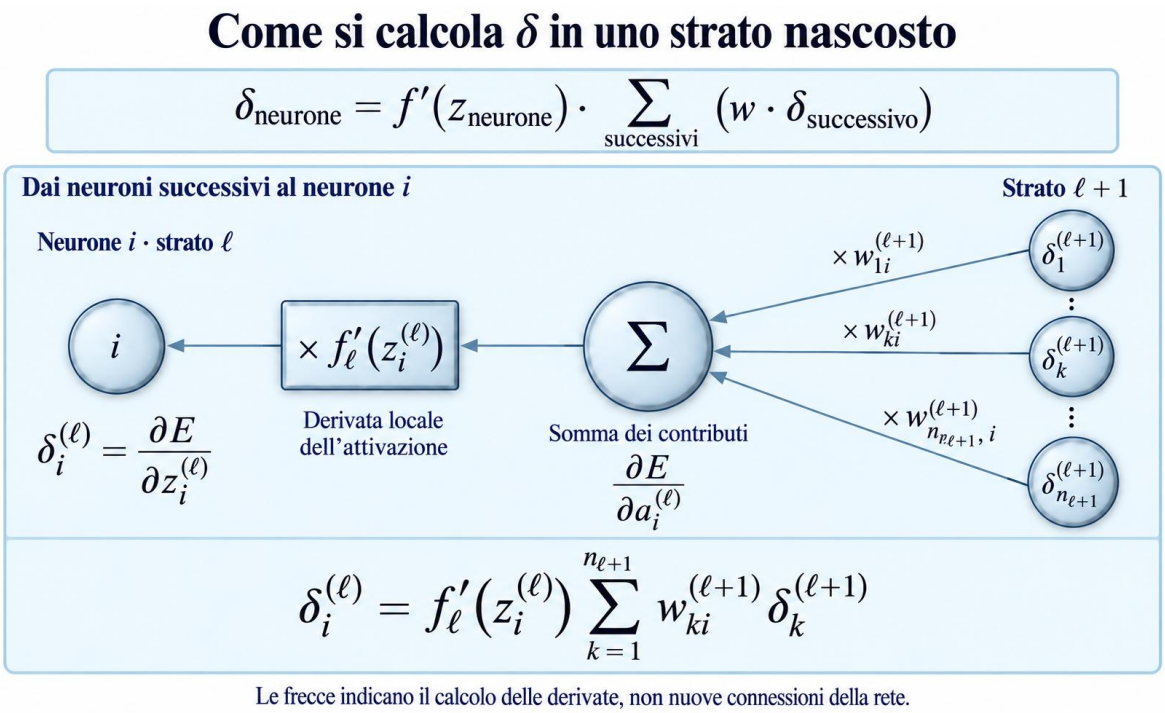


Figura 5. Calcolo a ritroso di $\delta_i^{(\ell)}$ in uno strato nascosto. I contributi $w_{ki}^{(\ell+1)}\delta_k^{(\ell+1)}$ dei neuroni successivi si sommano in $\partial E/\partial a_i^{(\ell)}$; il prodotto per $f_\ell'(z_i^{(\ell)})$ dà $\delta_i^{(\ell)}$, come nella (11). È lo schema del neurone della Figura 3, percorso all'indietro.

5.5 Il procedimento per un singolo esempio

Per un singolo esempio, fissati input e bersaglio:

1. **Forward.** Si calcolano e si conservano tutti gli $z_i^{(\ell)}$ e gli $a_i^{(\ell)}$ mediante (1) e (2), con la softmax nello strato di uscita multiclasse (§3.2).
2. **Errore.** Si calcola E a partire dall'uscita della rete e dal bersaglio.
3. **Delta di uscita.** Si calcolano i $\delta_i^{(L)}$ in base alla funzione di errore e alla trasformazione di uscita. Nei casi trattati dall'applicazione si utilizza (10a), (10b) o (10c).
4. **Delta nascosti.** Si calcolano i $\delta_i^{(\ell)}$ dagli ultimi strati verso i primi applicando ricorsivamente (11).
5. **Derivate dei parametri.** Noti tutti i termini $\delta_i^{(\ell)}$, le derivate di E rispetto a pesi e bias si ottengono con (8) e (9).
6. **Aggiornamento.** Si ricavano le variazioni dei parametri mediante (7) e si applicano insieme.

Tutte le derivate devono riferirsi allo stesso stato della rete: per questo si calcolano prima le derivate di E rispetto a tutti i pesi e bias e soltanto dopo si applicano insieme gli aggiornamenti.

6. Come la funzione di errore influisce sul gradiente

6.1 Confronto con l'errore quadratico

Il caso binario permette di vedere il ruolo della funzione di errore usando una sola uscita $a = \sigma(z)$ e un bersaglio $y \in \{0,1\}$. La probabilità assegnata alla classe 1 è a e quella assegnata alla classe 0 è $1 - a$. L'entropia incrociata binaria, abbreviata in BCE dall'inglese *binary cross-entropy*, diventa quindi

$$E_{\mathrm{BCE}} = -y \ln a - (1 - y) \ln (1 - a).$$

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Il confronto è il seguente:

Funzione di errore	Derivata rispetto a z
$E_{\text{quad}} = \frac{1}{2}(a-y)^2$	$\delta = (a-y)a(1-a)$
$E_{\text{BCE}} = -y \ln a - (1-y)\ln(1-a)$	$\delta = a - y$

La BCE si ottiene dalla definizione generale di entropia incrociata (3) ponendo $p = (a, 1-a)$ e $t = (y, 1-y)$, con le componenti ordinate come classe 1 e classe 0. Anche il legame fra le trasformazioni di uscita è diretto:

$$\sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{e^z + e^0}.$$

La sigmoide restituisce quindi la prima componente della softmax su $(z, 0)$; la seconda vale $1-a$. Considerando z come unica variabile, dalla prima componente della relazione (10c) si ritrova $\delta = a - y$.

Con l'errore quadratico compare il fattore $a(1-a)$, derivata della sigmoide. Questo fattore tende a zero quando a si avvicina a 0 o a 1, anche se la previsione è sbagliata. Si dice che la sigmoide è *in saturazione* quando si trova in una di queste regioni quasi piatte del grafico: una variazione di z produce allora una variazione molto piccola dell'uscita a.

Per $y = 0$ si ha $|\delta_{\text{quad}}| = a^2(1-a)$; per $y = 1$, $|\delta_{\text{quad}}| = a(1-a)^2$. Lo studio del massimo di queste due cubiche nell'intervallo $[0,1]$ dà il limite $|\delta_{\text{quad}}| \leq 4/27 \approx 0{,}148$.

Per esempio, con $a = 0{,}99$ e $y = 0$ si ottiene

$$\delta_{\text{quad}} = 0{,}99 \cdot 0{,}99 \cdot 0{,}01 = 0{,}009801, \qquad \delta_{\text{BCE}} = 0{,}99.$$

In questa situazione la derivata con entropia incrociata è circa cento volte maggiore. Quando a tende a 0 con $y = 1$, la derivata ottenuta con l'errore quadratico tende a 0, mentre quella ottenuta con l'entropia incrociata binaria tende a -1: l'entropia incrociata evita dunque che la saturazione della sigmoide di uscita attenui il segnale proprio in una previsione fortemente errata. Lo stesso accade nel caso multiclasse: quando p_c tende a zero, $\delta_c^{(L)}$ tende a -1. Questo, tuttavia, non elimina eventuali attenuazioni del gradiente negli strati nascosti.

Le due funzioni differiscono anche nella scala dei valori. Nel caso binario appena definito, $E_{\text{quad}} \leq \frac{1}{2}$, mentre se invece si usa l'errore quadratico su un intero vettore multiclasse, nella forma $\frac{1}{2}\sum_i (p_i - t_i)^2$ con bersaglio one-hot, il limite superiore è 1. L'entropia incrociata invece non è superiormente limitata.

Il diverso limite superiore dell'errore quadratico dipende dalla rappresentazione scelta. Con due classi, $p = (a, 1-a)$ e $t = (y, 1-y)$, la somma dei due scarti quadratici dimezzata vale $(a-y)^2$: è il doppio della loss scalare $\frac{1}{2}(a-y)^2$. I limiti 1 e $1/2$ sono quindi coerenti; non dipendono da una differenza sostanziale tra due e più classi.

Nell'applicazione l'entropia incrociata è usata per la classificazione multiclasse, mentre per quella binaria è mantenuto l'errore quadratico, come semplificazione didattica. Entrambe le possibilità sono richiamate nel §5.3.1.

6.2 Un esempio con tre classi

Riprendiamo l'esempio usato nel §3.2 e nella Guida alle attivazioni dell'applicazione:

$$z^{(L)} = (2;\ 0;\ -1).$$

La softmax produce

$$p \approx (0{,}843795;\ 0{,}114195;\ 0{,}042010).$$

L'errore dipende dalla classe osservata:

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

- **se la classe corretta è la prima**, il bersaglio è $t = (1; 0; 0)$:

$$E = -\ln p_1 \approx 0{,}170, \qquad \delta^{(L)} \approx (-0{,}156;\ 0{,}114;\ 0{,}042).$$

- **se la classe corretta è la terza**, il bersaglio è $t = (0; 0; 1)$:

$$E = -\ln p_3 \approx 3{,}170, \qquad \delta^{(L)} \approx (0{,}844;\ 0{,}114;\ -0{,}958).$$

Nel secondo caso, la rete ha assegnato soltanto il 4,2 % circa alla classe corretta. Poiché il bersaglio è $t_3 = 1$, si ha $\delta_3 = p_3 - 1 \approx -0{,}958$, vicino al valore limite −1 a cui δ_3 tende quando p_3 tende a zero: come discusso nel §5.3.3, la derivata non si attenua proprio quando la previsione è fortemente errata.

7. Dal singolo esempio al dataset

7.1 Gradiente medio e campionamento

L'obiettivo dell'allenamento è l'errore medio $\overline{E}(\theta)$ introdotto nel §4. Le derivate appena calcolate per un singolo esempio permettono di ricavare quelle della media.

Per linearità della derivazione, il gradiente dell'errore medio rispetto ai parametri è la media dei gradienti $\nabla E_r(\theta)$ degli errori associati ai singoli esempi:

$$\nabla \bar{E}(\theta) = \frac{1}{N} \sum_{r=1}^{N} \nabla E_r(\theta).$$

Gradient descent sull'intero dataset. Si calcolano, nello stesso stato della rete, i gradienti degli errori associati a tutti gli esempi del dataset, se ne fa la media e si applica un solo aggiornamento:

$$\theta_{\text{nuovo}} = \theta - \eta\, \nabla \bar{E}(\theta).$$

Questa procedura è detta *batch gradient descent*.

Discesa stocastica del gradiente (SGD, *stochastic gradient descent*). È detta stocastica perché a ogni passo la direzione di aggiornamento dipende dalla scelta casuale di un esempio, anziché essere calcolata usando contemporaneamente l'intero dataset. Nel caso del campionamento indipendente, indichiamo con R l'indice dell'esempio estratto a un dato passo: a ogni passo R viene scelto uniformemente e indipendentemente tra $1, \dots, N$, e i parametri vengono aggiornati subito:

$$\theta_{\text{nuovo}} = \theta - \eta\, \nabla E_R(\theta).$$

Poiché ciascun indice ha probabilità $1/N$, a parametri fissati il valore atteso del gradiente $\nabla E_R(\theta)$ è:

$$\mathbb{E}_R[\nabla E_R(\theta)] = \sum_{r=1}^{N} \frac{1}{N}\, \nabla E_r(\theta) = \frac{1}{N} \sum_{r=1}^{N} \nabla\, E_r(\theta) = \nabla \bar{E}(\theta).$$

Il valore atteso è qui calcolato rispetto alla scelta casuale di R tra gli N esempi del dataset fissato. In questo senso, il gradiente dell'errore associato all'esempio estratto è uno stimatore non distorto del gradiente dell'errore medio. Di conseguenza, a parametri fissati, l'aggiornamento medio di un passo di SGD coincide con quello di un passo di *batch gradient descent* con lo stesso tasso η, anche se il singolo aggiornamento può essere diverso.

7.2 Epoche e mini-batch

Un'epoca, o giro completo, consiste in un passaggio attraverso tutti gli esempi di allenamento.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Random reshuffling. All'inizio di ogni giro si genera un nuovo ordine casuale $r_1, \dots, r_N$ degli esempi e poi li si utilizza tutti, una volta ciascuno: il campionamento avviene quindi senza reinserimento. È questo l'algoritmo adottato dall'applicazione.

Poiché i parametri cambiano dopo ciascun aggiornamento, ogni esempio viene elaborato nello stato lasciato da quelli precedenti. I gradienti di un giro non sono quindi tutti calcolati nello stesso punto e la loro media non coincide in generale con il gradiente medio calcolato all'inizio: non si può trasferire automaticamente al campionamento senza reinserimento l'argomento di indipendenza del §7.1.

Per un tasso di apprendimento piccolo si può quantificare la differenza rispetto al batch completo: il §D sviluppa il confronto al primo ordine, sotto opportune ipotesi di regolarità.

Mini-batch. Nella pratica si usa spesso una soluzione intermedia tra il batch gradient descent e l'aggiornamento su un singolo esempio. Per un gruppo B di m esempi si calcola

$$g_B(\theta) = \frac{1}{m} \sum_{r \in B} \nabla E_r(\theta)$$

e si aggiorna la rete usando $g_B(\theta)$. Se B è scelto uniformemente a parametri fissati, $g_B(\theta)$ è uno stimatore non distorto del gradiente dell'errore medio. Poiché media i contributi di più esempi, questa quantità varia in genere meno da un campionamento all'altro rispetto al gradiente dell'errore associato a un singolo esempio: costituisce quindi una stima meno rumorosa del gradiente dell'errore medio sull'intero dataset. Rispetto al batch completo, richiede meno calcolo per ciascun aggiornamento e permette di sfruttare efficacemente il calcolo parallelo. Nella pratica i mini-batch sono spesso ottenuti rimescolando il dataset all'inizio dell'epoca, dividendolo in gruppi e aggiornando i parametri dopo ciascun gruppo. L'aggiornamento su un singolo esempio corrisponde al caso $m = 1$; il batch completo corrisponde al caso $m = N$. L'applicazione usa $m = 1$ insieme al random reshuffling senza reinserimento durante l'epoca.

8. Dall'allenamento alla generalizzazione

8.1 La scala degli input

Le formule precedenti assumono che gli input siano già espressi numericamente. La scala di questi numeri influisce però sui calcoli: nel primo strato, per esempio, la derivata rispetto a un peso contiene il fattore x_j (equazione (8)). Input di ordini di grandezza molto diversi possono quindi produrre gradienti sbilanciati e rendere più difficile la scelta di un unico tasso di apprendimento.

L'applicazione riporta gli input su una scala comune mediante una trasformazione min–max. Se X varia in un intervallo $[X_{\min}, X_{\max}]$ di ampiezza positiva, si pone

$$x = \frac{X - X_{\min}}{X_{\max} - X_{\min}}.$$

Questa trasformazione affine manda gli estremi in 0 e 1. La scala può essere fissata dal dominio del problema, come nei casi predefiniti del simulatore, oppure ricavata dai soli dati di allenamento. In quest'ultimo caso gli stessi estremi vanno poi usati per i dati di verifica e per le nuove previsioni: ricalcolarli cambierebbe il significato degli input. Un nuovo dato esterno all'intervallo iniziale può dare un valore normalizzato fuori da $[0,1]$.

8.2 Perché inizializzare i pesi in modo casuale

Prima dell'allenamento occorre assegnare valori iniziali ai parametri. Se due neuroni nascosti con la stessa attivazione partissero in condizioni perfettamente simmetriche — stessi pesi in ingresso, stessi bias e stessi

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

pesi verso ciascun neurone successivo — produrrebbero le stesse attivazioni e riceverebbero gli stessi aggiornamenti. Resterebbero così duplicati durante l'allenamento.

Se tutti i pesi e i bias fossero inizialmente nulli, al primo calcolo della backpropagation le somme pesate nella formula (11) si annullerebbero: i termini δ degli strati nascosti sarebbero quindi nulli. Ciò non implica che tutti i pesi resterebbero a zero: con la sigmoide, per esempio, le attivazioni nascoste valgono inizialmente $1/2$ e i pesi dello strato di uscita possono già cambiare al primo aggiornamento. Resta però la simmetria: i neuroni di uno stesso strato nascosto continuerebbero a ricevere aggiornamenti identici e non si differenzierebbero.

L'inizializzazione casuale dei pesi rompe questa simmetria e permette ai neuroni di evolvere diversamente. Nell'applicazione i pesi sono inizializzati uniformemente tra -1 e 1 e i bias a zero: non è necessario rendere casuali anche i bias per ottenere la differenziazione. Nelle reti più grandi conta anche la scala dei pesi iniziali, scelta in relazione alle dimensioni degli strati e all'attivazione. Il seme fisso del simulatore consente di riprodurre la stessa inizializzazione e confrontare gli esperimenti.

8.3 Errore di allenamento ed errore su dati nuovi

La discesa del gradiente usa gli esempi disponibili per ridurre l'errore di allenamento. L'interesse del modello, però, è produrre previsioni utili anche su esempi nuovi dello stesso problema: questa capacità è detta generalizzazione. Un errore di allenamento piccolo, da solo, non la garantisce.

Per valutarla si riserva un insieme di esempi alla verifica, senza usarli negli aggiornamenti dei pesi. Se l'errore di allenamento continua a diminuire mentre quello di verifica aumenta, è possibile che il modello si stia adattando anche alle particolarità del campione di allenamento: si parla di sovradattamento, o overfitting. Le curve del simulatore permettono di osservare questa distinzione; una singola oscillazione, tuttavia, non basta a diagnosticare il fenomeno.

Se i dati di verifica guidano ripetutamente la scelta dell'architettura o del momento in cui fermarsi, entrano indirettamente nel processo di scelta del modello. Per una valutazione finale indipendente servirebbero allora ulteriori dati di test. La prestazione osservata resta inoltre legata al tipo di dati su cui la si misura: dati nuovi molto diversi da quelli di allenamento possono richiedere un modello diverso.

9. Dalle formule a indici alla forma matriciale

Le formule a indici si raccolgono in vettori e matrici scrivendo simultaneamente le stesse equazioni per tutti i neuroni di uno strato.

Per uno strato $\ell = 1, \dots, L$ definiamo:

- $z^{(\ell)} \in \mathbb{R}^{n_\ell}$ è il vettore colonna dei valori pre-attivazione.
- $a^{(\ell)} \in \mathbb{R}^{n_\ell}$ è il vettore colonna delle attivazioni in uscita.
- $a^{(0)} = x$ è il vettore colonna degli input, come nel §1.3.
- $b^{(\ell)} \in \mathbb{R}^{n_\ell}$ è il vettore colonna dei bias.
- $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ è la matrice dei pesi. La riga i-esima contiene i pesi delle connessioni che arrivano al neurone i dello strato ℓ; la colonna j-esima contiene i pesi delle connessioni che partono dal nodo j dello strato $\ell - 1$. In particolare, $\left(W^{(\ell)}\right)_{ij} = w_{ij}^{(\ell)}$.

9.1 Forward

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)},$$

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

$$a^{(\ell)} = f_\ell\left(z^{(\ell)}\right).$$

Nell'applicazione la funzione f_ℓ è applicata componente per componente, salvo il caso della softmax nello strato di uscita.

9.2 Backward negli strati nascosti

Per uno strato nascosto ($1 \le \ell \le L-1$), raccogliamo in un vettore colonna i termini $\delta_i^{(\ell)}$:

$$\delta^{(\ell)} = \begin{pmatrix} \delta_1^{(\ell)} \\ \vdots \\ \delta_{n_\ell}^{(\ell)} \end{pmatrix} \in \mathbb{R}^{n_\ell}.$$

Per la componente i-esima, la formula ricavata nel §5.4 è

$$\delta_i^{(\ell)} = f_\ell'\left(z_i^{(\ell)}\right) \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)}\, \delta_k^{(\ell+1)}.$$

La somma a destra è la componente i-esima del prodotto tra matrici $\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}$:

$$\left[\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}\right]_{i,1} = \sum_{k=1}^{n_{\ell+1}} w_{ki}^{(\ell+1)}\, \delta_k^{(\ell+1)}.$$

Scrivendo contemporaneamente tutte le componenti si ottiene quindi

$$\delta^{(\ell)} = \left(\left(W^{(\ell+1)}\right)^T \delta^{(\ell+1)}\right) \odot f_\ell'\left(z^{(\ell)}\right),$$

dove $\odot$ indica il prodotto componente per componente, detto anche prodotto di Hadamard.

9.3 Derivate rispetto ai parametri

Indichiamo con $\frac{\partial E}{\partial W^{(\ell)}}$ la matrice $n_\ell \times n_{\ell-1}$ definita da

$$\left[\frac{\partial E}{\partial W^{(\ell)}}\right]_{ij} = \frac{\partial E}{\partial w_{ij}^{(\ell)}}.$$

Per l'equazione (8) abbiamo

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}.$$

Il secondo membro dell'equazione (8) è l'elemento in posizione (i,j) del prodotto tra la matrice colonna $\delta^{(\ell)}$ e la matrice riga $\left(a^{(\ell-1)}\right)^T$. Possiamo quindi raccogliere tutte le derivate di E rispetto ai pesi nella matrice:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)}\left(a^{(\ell-1)}\right)^T.$$

Per i bias, definito $\frac{\partial E}{\partial b^{(\ell)}}$ come il vettore colonna di componenti $\partial E / \partial b_i^{(\ell)}$, dall'equazione (9), otteniamo l'uguaglianza vettoriale

$$\frac{\partial E}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

9.4 Estensione ai mini-batch

Fin qui $z^{(\ell)}$, $a^{(\ell)}$ e $\delta^{(\ell)}$ sono vettori relativi a un singolo esempio. Per un mini-batch di m esempi, affianchiamo i vettori $a^{(\ell-1)}$ come colonne di $A^{(\ell-1)} \in \mathbb{R}^{n_{\ell-1} \times m}$ e i vettori $\delta^{(\ell)}$ come colonne di $\Delta^{(\ell)} \in \mathbb{R}^{n_\ell \times m}$. Le colonne corrispondono agli stessi esempi, nello stesso ordine; i termini δ sono le derivate delle loss individuali. Tutti i valori sono calcolati con gli stessi parametri, prima dell'aggiornamento. La media dei gradienti rispetto a $W^{(\ell)}$ è allora

$$\frac{1}{m} \Delta^{(\ell)} (A^{(\ell-1)})^T.$$

L'elemento in posizione (i, j) di questa matrice è $\frac{1}{m} \sum_{r=1}^{m} \Delta_{ir}^{(\ell)} A_{jr}^{(\ell-1)}$: il prodotto somma i contributi dei singoli esempi e il fattore $1/m$ ne restituisce la media. Forward, backward e calcolo dei gradienti diventano così in larga parte operazioni tra matrici, particolarmente adatte all'esecuzione parallela su GPU.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Appendice A. Temperatura e distribuzione di Boltzmann

Si può introdurre un parametro $\tau > 0$, chiamato temperatura, dividendo le pre-attivazioni per τ prima di applicare la softmax [2]:

$$p_i(\tau) = \frac{e^{z_i/\tau}}{\sum_{j=1}^{K} e^{z_j/\tau}} = \frac{e^{(z_i - z_{\max})/\tau}}{\sum_{j=1}^{K} e^{(z_j - z_{\max})/\tau}}.$$

A logits fissati, il rapporto $\frac{p_i(\tau)}{p_j(\tau)} = e^{(z_i - z_j)/\tau}$ mostra il ruolo del parametro: abbassare la temperatura accentua le differenze; alzarla porta i rapporti verso 1 e rende la distribuzione più uniforme. Per ogni temperatura positiva l'ordine delle probabilità, e quindi la classe più probabile, resta lo stesso.

Per $\tau \to 0^+$, se il massimo è unico, la probabilità della classe corrispondente tende a 1 e le altre a 0. Se il massimo è raggiunto da s classi, ciascuna tende invece a $1/s$. Per $\tau \to \infty$, tutte le probabilità tendono a $1/K$. Questi limiti si intendono per un numero finito di classi e logits fissati.

Per $z = (2; 0; -1)$, ad esempio:

τ	$p_1(\tau)$	$p_2(\tau)$	$p_3(\tau)$
0,5	0,979629	0,017943	0,002428
1	0,843795	0,114195	0,042010
2	0,628532	0,231224	0,140244

Aumentando la temperatura, la rete assegna un peso meno predominante alla classe favorita rispetto alle alternative. Se si sceglie sempre l'uscita con probabilità massima, modificare la temperatura non cambia la scelta. Nella generazione di testo con i modelli linguistici (LLM), però, il token successivo — per esempio una parola o una sua parte — può essere estratto casualmente secondo le probabilità calcolate, eventualmente restringendo il campionamento ai candidati più probabili. Aumentare la temperatura rende la distribuzione meno concentrata e dà complessivamente più spazio alle alternative al token favorito: può quindi produrre continuazioni meno prevedibili, anche se, a parità di contesto, l'ordine delle probabilità resta invariato.

In meccanica statistica, un sistema in equilibrio con un ambiente alla temperatura assoluta $T > 0$ occupa uno stato i di energia ε_i con probabilità data dalla distribuzione di Boltzmann [3]:

$$P_i(T) = \frac{e^{-\varepsilon_i/(k_B T)}}{Z(T)}, \qquad Z(T) = \sum_j e^{-\varepsilon_j/(k_B T)}.$$

Qui k_B è la costante di Boltzmann e $Z(T)$, detta funzione di partizione, è la somma che normalizza le probabilità. Ogni indice identifica uno stato; più stati possono avere la stessa energia.

La forma coincide con quella della softmax con temperatura: scelta una scala di energia $\varepsilon_0 > 0$, basta porre $z_i = -\varepsilon_i/\varepsilon_0$ e $\tau = k_B T/\varepsilon_0$. Alle pre-attivazioni maggiori corrispondono quindi energie minori. Una temperatura bassa favorisce gli stati di energia minima; una temperatura più alta attenua il peso delle differenze di energia.

Nella rete, tuttavia, i logits sono valori calcolati dal modello e τ è un parametro matematico, senza un necessario significato termodinamico.

La relazione $\delta = p - t$ si riferisce a $\tau = 1$. Con la softmax a temperatura τ di questa appendice e bersaglio fissato, la stessa derivazione dà $\frac{\partial E}{\partial z_i} = \frac{p_i(\tau) - t_i}{\tau}$. Cambiano quindi sia le probabilità sia il fattore che compare nel gradiente.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Appendice B. Entropia e informazione

B.1 Informazione e sorpresa

Un evento a cui attribuiamo una probabilità alta è atteso; uno a cui attribuiamo una probabilità bassa è inatteso: in teoria dell'informazione questa idea viene espressa assegnando all'osservazione di un evento di probabilità p il valore $-\ln p$, detto *informazione dell'evento* o *sorpresa*. Qui "sorpresa" indica dunque una quantità matematica, calcolata a partire dalla probabilità attribuita all'evento. Vale 0 per un evento certo e aumenta al diminuire della sua probabilità.

La scelta del logaritmo permette inoltre di sommare l'informazione di eventi indipendenti: se due eventi hanno probabilità p e p', la probabilità che si verifichino entrambi è pp' e la sorpresa associata al loro verificarsi insieme è quindi

$$-\ln(pp') = -\ln p - \ln p'.$$

La penalità $-\ln p_c$ introdotta nel §3.4 coincide quindi con la sorpresa associata all'etichetta osservata secondo le probabilità del modello. Ridurre l'entropia incrociata media equivale a ridurre la sorpresa media associata alle etichette degli esempi di allenamento. Il motivo del nome "entropia incrociata" è discusso nel prossimo §B.2.

B.2 Entropia di Shannon e divergenza di Kullback–Leibler

Per dare significato al nome *entropia incrociata*, consideriamo un esperimento con K possibili esiti. In questo approfondimento $t = (t_1, \dots, t_K)$ indica una distribuzione generica: l'esito q si verifica con probabilità t_q.

Se conosciamo queste probabilità, all'osservazione dell'esito q associamo l'informazione, o sorpresa, $-\ln t_q$, nel senso definito al §B.1. Per calcolare la sorpresa media, pesiamo ogni valore $-\ln t_q$ con la probabilità t_q che quell'esito si verifichi. Questa media ponderata è l'*entropia di Shannon*:

$$H(t) = -\sum_{q=1}^{K} t_q \ln t_q.$$

L'entropia rappresenta quindi l'informazione media fornita dall'osservazione dell'esito. La stessa quantità misura l'incertezza prima dell'osservazione: se l'esito è già certo, osservarlo non fornisce nuova informazione e l'entropia è nulla.

Un esito con probabilità zero non contribuisce alla media: nella formula si adotta la convenzione $0 \ln 0 = 0$, coerente con il limite $u \ln u \to 0$ per $u \to 0^+$.

Per esempio, una moneta equilibrata ha due esiti, testa e croce, ciascuno con probabilità $1/2$. La sorpresa associata a ciascun esito è $-\ln(1/2) = \ln 2$, perciò anche la sorpresa media è $H(t) = \ln 2 \approx 0{,}693$.

Supponiamo ora che gli esiti continuino a verificarsi con probabilità t_q, ma che un modello assegni loro probabilità p_q. Quando si osserva l'esito q, la sorpresa calcolata dal modello è $-\ln p_q$. Nel farne la media $H(t, p)$ usiamo invece ancora i pesi t_q, perché sono questi a descrivere quanto spesso ci aspettiamo che si presentino i diversi esiti, da cui il nome *entropia incrociata*:

$$H(t, p) = -\sum_{q=1}^{K} t_q \ln p_q.$$

Anche in questa somma gli esiti con $t_q = 0$ non contribuiscono e i relativi termini si omettono. Se il modello attribuisce probabilità zero a un esito che può verificarsi, l'entropia incrociata è infinita, ma, come abbiamo visto nel §3.4, con la softmax e pre-attivazioni finite questo caso non si presenta, se non per effetto degli arrotondamenti numerici (Appendice C).

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

Nell'esempio della moneta equilibrata, immaginiamo un modello che assegni probabilità $0{,}9$ a testa e $0{,}1$ a croce. Il modello considera testa poco sorprendente e croce molto sorprendente; la moneta, però, produce i due esiti con uguale probabilità. La sorpresa media del modello è quindi

$$H(t,p) = -\frac{1}{2}\ln 0{,}9 - \frac{1}{2}\ln 0{,}1 \approx 1{,}204.$$

Rispetto al modello che usa le probabilità corrette, nell'esempio la sorpresa media è aumentata di circa $1{,}204 - 0{,}693 = 0{,}511$.

In generale, la differenza

$$D_{\mathrm{KL}}(t||p) = \sum_{q:t_q>0} t_q \ln\frac{t_q}{p_q} = H(t,p) - H(t)$$

è detta *divergenza di Kullback–Leibler*. Misura quindi di quanto cresce la sorpresa media quando usiamo le probabilità p al posto delle probabilità t dell'esperimento. Questa differenza è sempre non negativa e vale zero soltanto quando $p = t$.

Nel caso della rete, il bersaglio t del singolo esempio è one-hot, la cui entropia è nulla: l'unico termine da considerare, infatti, è $-1\ln 1 = 0$. Di conseguenza,

$$E = H(t,p) = D_{\mathrm{KL}}(t||p) = -\ln p_c.$$

Ridurre l'errore del singolo esempio equivale quindi a ridurre la divergenza tra la distribuzione bersaglio e quella prevista dalla rete.

Con il logaritmo naturale, l'entropia $H(t)$, l'entropia incrociata $H(t,p)$ e la divergenza di Kullback–Leibler $D_{\mathrm{KL}}(t||p)$ si misurano in *nat*. Usando la base 2 si misurano in *bit* e tutti i valori vengono divisi per $\ln 2$. I punti di minimo e le direzioni dei gradienti restano gli stessi; cambia la loro scala e, a parità di tasso di apprendimento, anche l'ampiezza degli aggiornamenti.

Appendice C. Il calcolo numerico dell'entropia incrociata

Come abbiamo segnalato nel §3.3, nel calcolo in virgola mobile, una probabilità molto piccola può essere arrotondata a zero (underflow), anche usando la softmax con sottrazione del massimo. L'applicazione evita il logaritmo di zero usando una soglia minima μ per il suo argomento:

$$E_{\text{calcolato}} = -\ln\left(\max\left(p_c, \mu\right)\right).$$

La soglia è la costante JavaScript `Number.MIN_VALUE`, il minimo numero positivo rappresentabile: nei comuni browser $\mu = 2^{-1074}$ [4]. Ogni valore positivo rappresentabile è già almeno pari a μ; la sostituzione riguarda quindi soltanto lo zero numerico. L'errore calcolato non supera $-\ln\mu = 1074\ln 2 \approx 744{,}440$. Questo accorgimento mantiene finito il risultato, ma non recupera la probabilità persa nell'arrotondamento.

La soglia interviene soltanto nel valore dell'errore. La backpropagation usa $\delta = p - t$, ricavata nel §5.3.3 per l'entropia incrociata teorica e valutata con le probabilità calcolate numericamente. Quando interviene la soglia, il gradiente usato non è quindi la derivata della funzione troncata.

Si può evitare il passaggio per la probabilità calcolando l'errore direttamente dalle pre-attivazioni: con la notazione del §3.3, sostituendo la softmax in $E = -\ln p_c$ e applicando le proprietà del logaritmo si ottiene

$$E = -\ln p_c = (z_{\max} - z_c) + \ln\left(\sum_{i=1}^{K} e^{z_i - z_{\max}}\right).$$

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

La somma dentro il logaritmo è compresa tra 1 e K: ogni esponenziale è al massimo 1 e almeno uno vale 1. Questa forma evita il logaritmo di zero dovuto all'underflow di p_c.

Per chiarezza didattica, l'applicazione mantiene il calcolo da p_c con la soglia descritta sopra. L'alternativa del calcolo a partire dalle pre-attivazioni è presente nel codice come funzione commentata `crossEntropyFromLogits`.

Appendice D. Un giro di random reshuffling al primo ordine

Riprendiamo il confronto del §7.2. Sia $r_1, \dots, r_N$ l'ordine casuale degli esempi durante un giro e sia θ_0 il vettore dei parametri prima del primo aggiornamento. Useremo $O(\eta)$ per indicare una quantità la cui norma è limitata da una costante per η, quando η tende a zero; analogamente per $O(\eta^2)$. Il numero N di esempi resta fissato.

Se durante il giro i parametri restassero fissi al valore iniziale θ_0, la media dei gradienti sarebbe esattamente il gradiente dell'errore medio, qualunque sia l'ordine degli esempi:

$$\frac{1}{N}\sum_{k=1}^{N} \nabla E_{r_k}(\theta_0) = \frac{1}{N}\sum_{r=1}^{N} \nabla E_r(\theta_0) = \nabla \bar{E}(\theta_0).$$

Poiché ogni permutazione contiene ciascun esempio esattamente una volta, sotto questa ipotesi la media dei gradienti del giro coincide esattamente col gradiente medio del dataset.

Invece, nel random reshuffling con aggiornamento dopo ogni esempio, come nell'applicazione, i parametri cambiano durante il giro. Se θ_k indica il valore dei parametri dopo il k-esimo aggiornamento, allora

$$\theta_k = \theta_{k-1} - \eta\, \nabla E_{r_k}(\theta_{k-1}).$$

Definiamo il gradiente medio effettivamente usato durante il giro:

$$g_{\text{giro}} = \frac{1}{N}\sum_{k=1}^{N} \nabla E_{r_k}(\theta_{k-1}).$$

Per confrontarlo con $\nabla \bar{E}(\theta_0)$ osserviamo anzitutto che, dopo $k-1$ aggiornamenti,

$$\theta_{k-1} - \theta_0 = -\eta \sum_{s=1}^{k-1} \nabla E_{r_s}(\theta_{s-1}).$$

A numero di esempi fissato, se i gradienti restano limitati nell'intorno percorso dai parametri, la somma contiene un numero finito di termini e quindi

$$\theta_{k-1} - \theta_0 = O(\eta).$$

Se inoltre il gradiente di ciascun E_r varia in modo sufficientemente regolare con i parametri — per esempio è localmente lipschitziano — una variazione $O(\eta)$ dei parametri produce una variazione $O(\eta)$ anche del gradiente:

$$\nabla E_{r_k}(\theta_{k-1}) = \nabla E_{r_k}(\theta_0) + O(\eta).$$

Mediando sugli N aggiornamenti del giro, sempre a N fissato, si ottiene

$$g_{\text{giro}} = \frac{1}{N}\sum_{k=1}^{N} \nabla E_{r_k}(\theta_0) + O(\eta) = \nabla \bar{E}(\theta_0) + O(\eta).$$

Quindi, il gradiente medio effettivamente usato durante un giro differisce dal gradiente dell'errore medio calcolato all'inizio del giro di una quantità di ordine $O(\eta)$.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.

L'aggiornamento complessivo del giro è

$$\theta_N = \theta_0 - N\eta\, g_{\text{giro}} = \theta_0 - N\eta\, \nabla\bar{E}(\theta_0) + O(\eta^2).$$

Lo spostamento principale dei parametri è di ordine η, mentre la differenza rispetto al corrispondente passo batch è di ordine η^2. Pertanto, con η sufficientemente piccolo, un giro completo di random reshuffling con aggiornamento dopo ogni esempio ha, al primo ordine, lo stesso aggiornamento di un passo di batch gradient descent sull'errore medio con tasso di apprendimento $N\eta$. Il fattore N compare perché durante un giro si compiono N aggiornamenti, ciascuno con tasso di apprendimento η, mentre il batch gradient descent usa il gradiente medio. Il rimescolamento e gli aggiornamenti intermedi determinano le correzioni di ordine superiore e quindi la traiettoria esatta.

Con la ReLU, l'ipotesi di regolarità è soddisfatta, per esempio, in un intorno dei parametri iniziali in cui tutte le pre-attivazioni dei neuroni ReLU, per gli esempi considerati, restano diverse da zero. Il confronto non si applica automaticamente ai passi che attraversano un punto di non derivabilità.

Appendice E. Corrispondenza con il codice dell'applicazione

La struttura degli array usati nel nucleo dell'applicazione corrisponde direttamente alla notazione matematica. Tuttavia, nella dispensa gli indici dei neuroni partono da 1, mentre gli array JavaScript partono da 0; perciò nella tabella gli indici dei neuroni compaiono come i−1 e j−1.

Matematica	Array nell'applicazione
$w_{ij}^{(\ell)}$	weights[ℓ−1][i−1][j−1]
$b_i^{(\ell)}$	bias[ℓ−1][i−1]
$z_i^{(\ell)}$	zs[ℓ][i−1]
$a_i^{(\ell)}$	as[ℓ][i−1]
$\delta_i^{(\ell)}$	delta[ℓ][i−1]

In particolare, `as[0]` contiene gli input; `zs[ℓ]` e `delta[ℓ]` corrispondono agli strati $\ell = 1, \ldots, L$. Gli array `weights[ℓ−1]` e `bias[ℓ−1]` rappresentano invece il passaggio dallo strato $\ell - 1$ allo strato ℓ.

Riferimenti per gli approfondimenti

[1] P. Blanchard, D. J. Higham e N. J. Higham, Accurately computing the log-sum-exp and softmax functions, IMA Journal of Numerical Analysis 41(4), 2311–2330, 2021; preprint: arXiv:1909.03469. Analisi delle formule con sottrazione del massimo.

[2] G. Hinton, O. Vinyals e J. Dean, Distilling the Knowledge in a Neural Network, §§2–2.1. Softmax con temperatura e gradiente.

[3] D. Tong, Statistical Physics, §§1.3–1.3.1. Distribuzione di Boltzmann e funzione di partizione.

[4] MDN Web Docs, Number.MIN_VALUE. Valore minimo positivo nei comuni motori JavaScript.

[5] I. Goodfellow, Y. Bengio e A. Courville, Deep Learning, capp. 3, 4, 5, 6 e 8. Riferimento generale per teoria dell'informazione, calcolo numerico, generalizzazione, reti feed-forward e allenamento.

© Francesco Operetto, 2026 · Licenza CC BY-SA 4.0
Versione non definitiva: potrebbe contenere imprecisioni o errori. Commenti e segnalazioni sono benvenuti.
Ultimo aggiornamento: 18 ottobre 2026.
Testo elaborato e revisionato anche con strumenti di intelligenza artificiale.